Vehicle Rental Management System - OOP Report

1. Overview

The system is split into three packages. The `model` package holds the domain classes (`Vehicle`, `Car`, `Motorcycle`, `Truck`, `Customer`, `Rental`) and the `Maintainable` interface. The `service` package has `RentalSystem`, which stores everything in `ArrayLists` and enforces all the business rules, and `RentalException`, a checked exception used when a rule is broken. The `app` package only deals with the console: `ConsoleMenu` reads input and prints results, `InputHelper` validates typed values, and `Main` just starts the menu. This keeps the business logic out of `Main`, as required. A separate class, `ScenarioTest`, runs all 17 test scenarios from the assignment, and all of them pass.

2. Abstraction and abstract classes

Abstraction means showing what an object can do and hiding how it does it. In this project the rest of the program only needs to know that every vehicle can calculate a rental cost and display its information; it does not care how a truck adds its service fee or how a car adds insurance.

`Vehicle` is declared `abstract` because a "general vehicle" is never rented, only a car, a motorcycle or a truck. It contains everything the three types share: the ID, brand, model, year, daily price, availability, the maintenance flag and a counter of rentals since the last service. These fields are `private`, and they are only changed through methods that validate the data (for example `setDailyPrice` rejects zero or negative prices). It also gives shared helper methods such as `baseCost(days)`, which returns `dailyPrice × days` and rejects non-positive days.

At the same time `Vehicle` declares three abstract methods: `calculateRentalCost(int days, boolean withOptionalService)`, `displayInfo()` and `getType()`. Every subclass must implement them, so the compiler guarantees that no vehicle type can be added without its own pricing and display.

3. Abstract classes vs interfaces

Both are used in the project, but for different reasons.

	Abstract class (`Vehicle`)	Interface (`Maintainable`)
Purpose	Describes what a vehicle **is**	Describes something a vehicle **can do**
State	Has fields (ID, price, flags…)	No instance fields
Constructors	Yes, called with `super(...)`	No
Code	Concrete + abstract methods	Only method signatures here
Inheritance	A class can extend only one	A class can implement many

`Vehicle` is an abstract class because the three types share a lot of data and code, and writing it once avoids repetition. `Maintainable` is an interface because maintenance is a capability, not an identity. `Car`, `Motorcycle` and `Truck` each declare `implements Maintainable` and provide their own behaviour:

- **Car**: oil change, tire rotation and brake check; due after 5 rentals.

- **Motorcycle**: chain lubrication and tire pressure, plus helmet cleaning if it has one; due after 3 rentals, or 2 for bikes of 1000cc or more.
- **Truck**: brakes, suspension and hydraulics, plus an axle load test for heavy trucks; due after 4 rentals, or 2 for trucks of 10 tons or more.

In all three, `needsMaintenance()` returns true if staff flagged the vehicle or if the rental count reached the limit, and `performMaintenance()` resets both. Because `Vehicle` does not implement the interface itself, another vehicle type that cannot be maintained could still be added later without changing the interface.

4. Polymorphism

4.1 Inheritance-based (runtime) polymorphism

`RentalSystem` stores all vehicles in one `ArrayList<Vehicle>`. When a rental is created, `Rental` calls `vehicle.calculateRentalCost(days, optionalService)` on a `Vehicle` reference, and Java chooses the right version at runtime:

- `Car`: `baseCost(days) + 15 × days` if insurance is chosen → $60, 4 days with insurance = **$300**
- `Motorcycle`: `baseCost(days) + 5 × days` if a helmet is chosen
- `Truck`: `baseCost(days) + 50` → $100, 3 days = **$350**

The same happens with `displayInfo()` when listing vehicles and with `getOptionalService()`, which the menu uses to decide which question to ask ("Add Insurance?" / "Add Helmet?" / nothing for a truck). So there is no `if (type == "Car")` or `switch` anywhere in the cost logic. Adding a new vehicle type would only need a new subclass.

4.2 Interface-based polymorphism

For maintenance the system works with `Maintainable` references instead of `Vehicle` references. It first collects every vehicle that is `instanceof Maintainable` into a `List<Maintainable>`, then calls `needsMaintenance()` on each item. `getVehiclesNeedingMaintenance()` returns a `List<Maintainable>`, and `performMaintenance(id)` calls `maintainable.performMaintenance()`. The calling code does not know whether it is a car, motorcycle or truck, but each one prints and does its own service steps. The same reference type is also used in `rentVehicle` to block a vehicle that needs maintenance, and test 17 in `ScenarioTest` puts all three types in a `List<Maintainable>` to show that the correct subclass method runs.

5. Overriding

Overriding is when a subclass gives its own version of a method that already exists in the parent, with the same signature. It is resolved at runtime. Examples:

- `calculateRentalCost(int, boolean)`, `displayInfo()` and `getType()`: abstract in `Vehicle`, overridden in all three subclasses.
- `getOptionalService()`: `Vehicle` returns `null` (no extra service), `Car` and `Motorcycle` override it.
- `performMaintenance()` and `needsMaintenance()`: implemented from the interface in each subclass.
- `toString()`: overridden in `Vehicle`, `Customer` and `Rental` for readable output.

All of them use `@Override`, so the compiler catches spelling mistakes in the signature.

6. Overloading

Overloading is having several methods with the same name but different parameters in the same class. It is resolved at compile time.

- **Search:** `findVehicles(String keyword)` searches by brand or model, while `findVehicles(double maxDailyPrice)` searches by budget. `findVehicle(String id)` returns a single vehicle.
- **Cost:** `calculateRentalCost(int days)` and `calculateRentalCost(int days, boolean withOptionalService)` in `Vehicle`; the short one calls the full one with `false`.
- **Renting:** `rentVehicle(cId, vId, days)` and `rentVehicle(cId, vId, days, optionalService)`.
- **Input:** `readInt(prompt)` and `readInt(prompt, min, max)` in `InputHelper`.
- **Constructors:** `Car` has a full constructor and a short one that defaults to 5 seats and automatic, using `this(...)`. `Motorcycle`, `Customer` (without email) and `Rental` also have overloaded constructors.

7. Other keywords and design points

- **this**: used to set fields in constructors and to chain constructors (`this(id, brand, ..., 5, Transmission.AUTOMATIC)`).
- **super**: each subclass constructor calls `super(vehicleId, brand, model, year, dailyPrice)` so validation happens in one place.
- **static**: `Rental.nextNumber` generates unique rental IDs shared by all rentals, and the fee constants belong to the class because every vehicle of a type charges the same fee.
- **final**: fee constants (`INSURANCE_PER_DAY`, `HELMET_PER_DAY`, `SERVICE_FEE`), fields that must never change after creation (IDs, license number, the customer and vehicle of a rental), and validated setters called from constructors.
- **Association**: a `Rental` holds a reference to one `Customer` and one `Vehicle`; a customer keeps a list of their rentals, which gives the rental history and the vehicles currently rented.
- **Exceptions**: model classes throw `IllegalArgumentException/IllegalStateException` for invalid data; `RentalSystem` throws `RentalException` for broken business rules; `ConsoleMenu` catches them and shows a message, so the program never stops unexpectedly. `InputHelper` keeps asking until a valid number is typed.